

Array Techniques

1. Array Order Reversal

Algorithm

Input: An integer array A and its size n.

Output: The reversed array REV.

Step 1: Start

Step 2: Read n

Step 3: Create an array A of size n

Step 4: for i=1 to n
 Read A[i]

End for

Step 5: for i=0 to n
 Display A[i]

End for

Step 6: Create an array REV of size n

Step 7: Read j=1

Step 8: for i=n-1 to 0
 REV[j]=A[i]
 j++

End for

Step 9: for i=0 to n
 Display REV[i]

End for

Step 10: Stop

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a[10],rev[10],n,j,i;

    printf("Enter the limit: ");
    scanf("%d",&n);

    printf("Enter the %d elements: ",n);
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
```

```
printf("Entered array elements are: ");
for(i=0;i<n;i++)
    printf("%d\n",a[i]);

j=0;

for(i=n-1;i>=0;i--)
{
    rev[j]=a[i];
    j++;
}

printf("Reversed array elements are: ");
for(i=0;i<n;i++)
    printf("%d\n",rev[i]);

getch();
}
```

Working of the Program:

- Declares integer arrays a and rev to store the original and reversed elements, respectively.
- Declares integer variables n for user limit, j for reversing the array and i for the for loop.
- Prompts the user to enter the desired array size (n).
- Reads the n elements from the user and stores them in the array a.
- Prints the entered elements.
- Initializes j to 0.
- Iterates through the array a in reverse order (from n-1 to 0).
- Copies each element from a to the corresponding index in rev, starting from the end of array a.
- Increments j after each copy.
- Prints the reversed elements stored in the rev array.

Key points to remember:

- The program first gets the array size from the user.
- It then reads the elements into the array a.
- To reverse the array, it iterates through the array a in reverse order.

- Each element from array a is copied to the corresponding index in rev, starting from the end of array a.
- This effectively reverses the order of the elements.
- Finally, the reversed elements are printed.

2. Finding the Maximum Number in a Set

Step 1: Start

Step 2: Read n

Step 3: for i=0 to n

 Input A[i]

Step 4: output A[i]

Step 5: for i=0 to n

 for j=i+1 to n

 if A[i]=A[j]

 for k=j to n

 A[k]=A[k+1]

 End for

 n=n-1;

 j=j-1;

 end if

 end for

end for

Step 6: output A[i]

Step 7: Stop

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
int a[10],rev[10],n,j,i;
```

```
printf("Enter the limit: ");
```

```
scanf("%d",&n);
```

```
printf("Enter the %d elements: ",n);
```

```
for(i=0;i<n;i++)
```

```
scanf("%d",&a[i]);
```

```
printf("Entered array elements are: ");
```

```
for(i=0;i<n;i++)
```

```
        printf("%d\n",a[i]);

    j=0;

    for(i=n-1;i>=0;i--)
    {
        rev[j]=a[i];
        j++;
    }

    printf("Reversed array elements are: ");
    for(i=0;i<n;i++)
        printf("%d\n",rev[i]);

    getch();
}
```

Working of the Program:

- Declares an integer array named a to store the original elements.
- Declares another integer array named rev to store the reversed elements.
- Declares an integer n to store the number of elements in the array.
- Declares integer variables i and j used as loop counters.
- Prompts the user to enter the number of elements and stores it in the variable n.
- Prompts the user to enter the elements of the array.
- For loop reads n elements from the user and stores them in the array a.
- One more For loop prints each element of the original array a on a new line.
- Initializes the index j for the rev array to 0. This index will be used to keep track of the position in the rev array.
- The first For loop iterates backward through the original array a.
- In each iteration, the current element of a is assigned to the corresponding position in the rev array. This effectively reverses the order of the elements.
- The index j for the rev array is incremented to move to the next position.
- Using a For Loop it prints the Reversed Array

Key Points to remember:

- Takes an array of integers as input from the user.
- Creates a new array to store the reversed elements.
- Iterates through the original array in reverse order.
- Assigns each element from the original array to the new array in reverse order.
- Prints the reversed array.

3. Removal of Duplicates from an Ordered Array

Step 1: Start

Step 2: Read n

Step 3: for to n

Input A[i]

Step 4: output A[i]

Step 5: for i=0 to n

for j=i+1 to n

if A[i]==A[j]

count=count+ 1;

break;

Step 6: Display Count

Step 7: Stop

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a[10],n,j,i,count=0;

    printf("Enter the limit: ");
    scanf("%d",&n);

    printf("Enter the %d elements: ",n);
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    printf("Entered array elements are: ");
    for(i=0;i<n;i++)
        printf("%d\\n",a[i]);
```

```
for(i=0;i<=n;i++)
{
    for(j=i+1;j<n;j++)
    {
        if(a[i]==a[j])
        {
            count++;
            break;
        }
    }
}

printf("The number of duplicates are: %d",count);
getch();
}
```

Working of the Program:

- Declare an integer array named a to store the input elements.
- Declare Integer variables i and j used as loop counters.
- Declare an integer variable Count to keep track of the number of duplicate elements found.
- Prompts the user to enter the number of elements and the elements entered by the user is stored in the variable n.
- Prompts the user to enter the elements of the array.
- A For loop reads n elements from the user and stores them in the array a.
- A For loop prints each element of the original array a on a new line.
- The outer For loop iterates through each element in the array.
- The inner For loop compares the current element (a[i]) with all subsequent elements (a[j]).
- If a duplicate is found, Increment the count variable to indicate that a duplicate has been found.
- Once a duplicate is found and count variable is incremented, Exit the inner loop since we have already found a duplicate for the current element a[i].
- Prints the total number of duplicate elements found.

Key Points to remember:

- It takes an array of integers as input from the user.
- It uses nested loops to compare each element with all subsequent elements.

- If a duplicate is found, the count is incremented.
- The program prints the total number of duplicate elements found.

4. Partitioning an Array

Step 1: Start

Step 2: Read n

Step 3: for i=0 to n
 Read A[i]

Step 4: Read mid

Step 5: for i=0 to mid
 Display A[i]

Step 6: for i=mid+1 to n
 Display A[i]

Step 6: Stop

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a[10],n,i,mid=0;

    printf("Enter the limit: ");
    scanf("%d",&n);

    printf("Enter %d elements: ",n);
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    mid=(0+(n-1))/2;

    printf("Mid=%d\n",mid);

    printf("First Partition: ");
    for(i=0;i<=mid;i++)
        printf("\n%d",a[i]);

    printf("Second Partition: ");
    for(i=mid+1;i<n;i++)
        printf("\n%d",a[i]);

    getch();
}
```

}

Working of the Program:

- Declare An integer array a to store the input elements
- Declare an integer n to store the number of elements entered by the user.
- Declare an integer i as a loop counter.
- Declare an integer mid to store the index of the middle element.
- Prompts the user to enter the number of elements and the elements entered by the user is stored in n.
- Calculates the index of the middle element in the array. This formula finds the average of the first and last indices.
- Print First Partition: Prints all elements from the beginning of the array up to the midpoint (inclusive).
- Print Second Partition: Prints all elements from the midpoint+1 to the end of the array.

Key Points to remember:

- Takes an array of integers as input.
- Calculates the midpoint index of the array.
- Divides the array into two partitions:
- The first partition includes elements from the beginning to the midpoint.
- The second partition includes elements from the midpoint+1 to the end.
- Prints the midpoint index and the elements in each partition.

5. Finding the kth Smallest Element

Step 1: Start

Step 2: Read n

Step 3: for i=0 to n

 Read A[i]

Step 4: Read smallest=A[0]

Step 5: for i=0 to n

 if(A[i]<smallest)

 smallest=a[i]

Step 6: Display smallest

Step 7: Stop


```
#include<stdio.h>
#include<conio.h>
void main()
{
    int n,i,smallest,a[30];

    printf("Enter the limit: ");
    scanf("%d",&n);

    printf("Enter the %d elements: ",n);
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    smallest=a[0];

    for(i=0;i<n;i++)
    {
        if(a[i]<smallest)
        {
            smallest=a[i];
        }
    }
    printf("Smallest Number in the given list is: %d",smallest);
    getch();
}
```

Working of the Program:

- Declare an integer array named a to store the input elements.
- Declare an integer n to store the number of elements in the array.
- Declare an integer variable i used as a loop counter.
- Declare an variable smallest to store the smallest element found in the array.
- Prompts the user to enter the number of elements and the elements entered by the user is stored in the variable n.
- Prompts the user to enter the elements of the array.
- A For loop reads n elements from the user and stores them in the array a.
- Initially, the smallest variable is assigned the value of the first element in the array (a[0]).
- A For loop iterates through each element in the array.
- If the current element (a[i]) is smaller than the current value stored in smallest:

- Updates the smallest variable to the current element.
- Displays the smallest number found in the array.

Key Points to remember:

- Takes an array of integers as input from the user.
- Assumes the first element as the smallest initially.
- Compares each element in the array with the current smallest value.
- If a smaller element is found, it updates the smallest value.
- Finally, it prints the smallest number found in the array.